EX12 - Movimiento vertical con funciones

De un script lineal a varias funciones que colaboran

1. Objetivo de la clase

Hasta ahora hemos escrito funciones relativamente independientes. En EX12 vamos a tomar el simulador vertical de Euler del EX08 y dividirlo en responsabilidades pequeñas. El objetivo principal no es mejorar la física: es aprender a diseñar un programa con varias funciones que se llaman entre sí.

Idea central: una función debería tener una responsabilidad clara y una interfaz sencilla.

• Separar validacion, integracion numerica, solucion analitica y errores.

• Evitar variables globales ocultas.

• Practicar varios valores de retorno.

• Entender como una funcion puede llamar a otra.

• Preparar el terreno para modulos y proyectos mas grandes.

2. Modelo fisico

Usaremos exactamente el mismo modelo del EX08:

```
dh/dt = v
dv/dt = -g
```

Con Euler explicito:

```
h_new = h + v * dt
v_new = v - g * dt
t_new = t + dt
```

La solucion analitica seguira sirviendo para validar el simulador:

```
h_max = h0 + v0**2 / (2*g)
t_f   = (v0 + sqrt(v0**2 + 2*g*h0)) / g
v_f   = -sqrt(v0**2 + 2*g*h0)
```

3. Arquitectura del programa

validate_inputs(...)	euler_step(...)	simulate_vertical_motion(...)	analytical_solution(...)	calculate_errors(...)
inputs -> valid/invalid	state k -> state k+1	repeat steps until impact	exact reference values	numerical vs exact

No es obligatorio usar exactamente estos nombres, pero si quiero que el problema quede dividido por responsabilidad. Una posible secuencia es:

```
main
  -> validate_inputs(...)
  -> simulate_vertical_motion(...)
       -> euler_step(...) many times
  -> analytical_solution(...)
  -> calculate_errors(...)
  -> print results
```

Importante: simulate_vertical_motion() puede llamar a euler_step(). Una funcion puede usar otra funcion exactamente igual que usa math.sqrt().

4. Responsabilidad sugerida de cada funcion

Funcion	Entradas	Salidas	Responsabilidad
validate_inputs	h0, v0, g, dt	bool / None	Comprobar condiciones fisicas y numericas
euler_step	h, v, t, g, dt	h_new, v_new, t_new	Avanzar exactamente un paso
simulate_vertical_motion	h0, v0, t0, g, dt	t_f, max_h, v_f, iterations	Ejecutar todo el while
analytical_solution	h0, v0, g	h_max, t_f, v_f	Calcular referencia exacta
calculate_errors	numerical, analytical	errores	Comparar resultados

5. Un solo paso: la funcion mas limpia

euler_step() es una funcion ideal para aprender diseño: no necesita while, no imprime y no conoce toda la trayectoria. Solo transforma un estado actual en el siguiente estado.

```
def euler_step(h, v, t, g, dt):
    # calculate next state
    ...
    return h_new, v_new, t_new
```

Esto puede leerse como una funcion matematica:

(h_k, v_k, t_k, g, dt) -> (h_{k+1}, v_{k+1}, t_{k+1})

Observa que devuelve tres valores. Ya sabes desempaquetarlos:

```
h, v, t = euler_step(h, v, t, g, dt)
```

6. Funcion de simulacion

simulate_vertical_motion() sera la funcion de mayor nivel. Inicializa el estado, ejecuta el while, llama a euler_step(), actualiza max_h e iterations, y devuelve los resultados finales.

```
def simulate_vertical_motion(h0, v0, t0, g, dt):
    # initialize state
    # while altitude >= 0:
    #     call euler_step(...)
    #     update max height
    #     update iterations
    # return results
```

Esta funcion no deberia calcular la solucion analitica. Esa es otra responsabilidad.

7. Programa principal y flujo de datos

El main debe empezar a parecerse a una receta de alto nivel, no a una coleccion de ecuaciones:

```
define inputs
validate inputs
run numerical simulation
compute analytical solution
compute errors
print results
```

Cuando el main se puede leer casi como pseudocodigo, normalmente la separacion de responsabilidades va por buen camino.

8. Casos de prueba

Caso	h0	v0	dt	Esperado
Nominal	0	50	0.1	Valid simulation
Smaller dt	0	50	0.01	Lower Euler error
Initial altitude	100	50	0.1	General analytical formulas
Invalid dt	0	50	0	Rejected
Invalid g	0	50	0.1	Rejected

9. Requisitos del ejercicio

• No usar variables globales dentro de las funciones.

• No usar scipy ni RK4.

• euler_step() debe realizar un unico paso.

• simulate_vertical_motion() debe contener el while.

• La solucion analitica debe vivir en una funcion distinta.

• Mantener return None o una politica clara para inputs invalidos.

• La salida final debe comparar numerico vs analitico.

• Puedes conservar prints de control durante desarrollo, pero intenta que las funciones de calculo no dependan de ellos.

10. Learning tracker

Concepto	EX12
Varias funciones colaborando	YES
Funcion llamando a otra funcion	YES
Separacion de responsabilidades	YES
Multiples valores de retorno	YES
Scope local	YES
Main de alto nivel	YES
Reutilizacion de Euler	YES
Preparacion para modulos	YES

Criterio de exito: si puedes leer el main sin ver las ecuaciones internas y aun asi entender el flujo completo del programa, el ejercicio esta bien estructurado.